
TRÍ TUỆ NHÂN TẠO

Machine Learning

Phần 4 — Regression Analysis

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	3
1 Khái niệm nền — Hồi quy trong học máy	4
2 Phân tích hồi quy (Regression Analysis)	4
2.1 Thuật ngữ	5
2.2 Cách hồi quy hoạt động	5
2.3 Các loại hồi quy trong học máy	5
2.4 Mô hình hồi quy đơn và đa biến	6
2.5 Chọn mô hình hồi quy	6
2.6 Metric đánh giá	6
2.7 Ứng dụng	6
2.8 Xây dựng regressor trong Python	7
3 Hồi quy tuyến tính (Linear Regression)	8
3.1 Dữ liệu ví dụ (một feature)	8
3.2 Quan hệ tuyến tính dương và âm	9
3.3 Hai loại hồi quy tuyến tính	9
3.3.1 Hồi quy tuyến tính đơn	10
3.3.2 Hồi quy tuyến tính đa biến	10
3.4 Cách hoạt động	10
3.5 Hàm giả thuyết	10
3.6 Đường khớp tốt nhất	11
3.7 Hàm mất mát	11
3.8 Gradient descent	11
3.9 Giả định mô hình	11
3.10 Metric đánh giá	12
3.11 Ứng dụng, ưu điểm và thách thức	12
4 Hồi quy tuyến tính đơn (Simple Linear Regression)	12
4.1 Cách hoạt động	13
4.2 Giả định	13
4.3 Triển khai Python	14
5 Hồi quy tuyến tính đa biến (Multiple Linear Regression)	17
5.1 Giả định	18
5.2 Triển khai Python	18
5.3 Ứng dụng và thách thức	19
5.4 So sánh đơn và đa biến	20
6 Hồi quy đa thức (Polynomial Regression)	20
6.1 Phương trình	21
6.2 Cách hoạt động	21
6.3 Triển khai Python	21

Lời nói đầu

Nguồn

Tài liệu biên soạn và dịch đầy đủ từ Tutorialspoint Machine Learning — mục Regression Analysis: https://www.tutorialspoint.com/machine_learning/machine_learning_regression_analysis.htm.

Cách dùng

- Đọc sau phần Statistics và Data Visualization.
- File dữ liệu mẫu: thư mục data/ (Salary_Data.csv, data.csv, ice_cream_selling_data.csv).
- Chạy code từ thư mục 4.RegressionAnalysis/.

1 Khái niệm nền — Hồi quy trong học máy

Phân tích hồi quy (Regression Analysis)

Kỹ thuật thống kê dự đoán **giá trị số liên tục** dựa trên quan hệ giữa biến độc lập và biến phụ thuộc. Mục tiêu: vẽ đường/đường cong khớp dữ liệu tốt nhất và ước lượng ảnh hưởng của biến này lên biến kia.

Học có giám sát — hồi quy

Đầu ra là **số liên tục**; mô hình học từ feature (độc lập) và target (phụ thuộc) để dự đoán trên dữ liệu mới.

Thuật ngữ thường gặp

- **Biến độc lập** (predictor, feature).
- **Biến phụ thuộc** (target, response).
- **Đường hồi quy**: đường/đường cong khớp dữ liệu.
- **Overfitting / Underfitting**: variance cao / bias cao.
- **Outlier, đa cộng tuyến** (multicollinearity).

Các chương trong phần này

Hồi quy tuyến tính (tổng quan), hồi quy tuyến tính đơn, hồi quy tuyến tính bội, hồi quy đa thức.

2 Phân tích hồi quy (Regression Analysis)

Phân tích hồi quy

Kỹ thuật thống kê dự đoán **giá trị số liên tục** dựa trên quan hệ giữa biến độc lập và biến phụ thuộc. Mục tiêu: vẽ đường/đường cong khớp dữ liệu tốt nhất và ước lượng ảnh hưởng của biến này lên biến kia.

Vai trò trong học máy

Hồi quy là **học có giám sát**: mô hình học từ feature (độc lập) và target (phụ thuộc, số liên tục) để dự đoán trên dữ liệu mới. Ứng dụng: dự báo, phân tích dự đoán, tối ưu kinh doanh, kinh tế, tài chính.

2.1 Thuật ngữ

Thuật ngữ thường gặp

- **Biến độc lập** (predictor, feature): dùng để dự đoán.
- **Biến phụ thuộc** (target, response): giá trị cần dự đoán.
- **Đường hồi quy**: đường/đường cong khớp điểm dữ liệu.
- **Overfitting**: khớp tốt tập huấn luyện, kém tập kiểm tra (phương sai cao).
- **Underfitting**: mô hình kém cả trên tập huấn luyện (độ lệch cao).
- **Outlier**: điểm cực cao/thấp, lệch pattern chung.
- **Đa cộng tuyến** (multicollinearity): feature phụ thuộc lẫn nhau.

2.2 Cách hồi quy hoạt động

Quy trình

Thuật toán hồi quy được huấn luyện trên tập có nhãn (feature + target). Trong huấn luyện, mô hình học quan hệ giữa predictor và target; sau đó dự đoán giá trị mới từ quan hệ đã học.

2.3 Các loại hồi quy trong học máy

Phân loại thường dựa trên: số biến độc lập, kiểu biến phụ thuộc, hình dạng đường hồi quy.

Các loại phổ biến

1. **Hồi quy tuyến tính** — quan hệ tuyến tính; gồm đơn biến và đa biến.
2. **Hồi quy logistic** — xác suất sự kiện; phân loại nhị phân (sigmoid).
3. **Hồi quy đa thức** — quan hệ bậc n ; bất phi tuyến, nhạy outlier.
4. **Lasso (L1)** — chuẩn hóa, giảm overfitting; dữ liệu chiều cao.
5. **Ridge (L2)** — chuẩn hóa; xử lý đa cộng tuyến, nhiều tham số.
6. **Cây quyết định hồi quy** — chia tập con, giảm MSE từng nút.
7. **Random Forest hồi quy** — tập hợp nhiều cây (bagging).
8. **SVR** — máy vectơ hỗ trợ cho hồi quy; chịu outlier, phi tuyến.

Hồi quy tuyến tính đơn

Một biến độc lập X dự đoán Y :

$$Y = mX + b$$

(Y phụ thuộc, X độc lập, m độ dốc, b hệ số chặn.)

Lỗi trong nguồn gốc

Bài gốc Tutorialspoint đôi chỗ ghi nhầm X/Y là biến phụ thuộc/độc lập; phần dưới dùng ký hiệu chuẩn.

2.4 Mô hình hồi quy đơn và đa biến

- **Mô hình đơn giản:** một feature duy nhất.
- **Mô hình đa biến:** nhiều feature.

2.5 Chọn mô hình hồi quy

Tiêu chí

So sánh MSE, MAE, R^2 ; độ phức tạp; khả năng giải thích. Chọn mô hình có metric tốt, đơn giản và dễ diễn giải khi có thể.

2.6 Metric đánh giá

Các chỉ số

- **MAE:** trung bình $|y - \hat{y}|$.
- **MSE:** trung bình $(y - \hat{y})^2$.
- **Median absolute error:** trung vị $|y - \hat{y}|$.
- **RMSE:** $\sqrt{\text{MSE}}$.
- R^2 : tối đa 1.0; có thể âm nếu mô hình rất kém.
- **MAPE:** MAE dạng phần trăm.

2.7 Ứng dụng

Ứng dụng điển hình

- **Dự báo:** GDP, giá dầu, chuỗi thời gian.
- **Tối ưu:** giờ cao điểm khách hàng.
- **Hiệu chỉnh quyết định:** đánh giá và sửa chính sách.
- **Kinh tế / tài chính:** cung-cầu, rủi ro danh mục.

2.8 Xây dựng regressor trong Python

Các bước (scikit-learn)

1. Import thư viện.
2. Nạp dữ liệu.
3. Chia train/test.
4. Huấn luyện và dự đoán.
5. Vẽ đồ thị.
6. Tính metric.

Hồi quy tuyến tính với dữ liệu tổng hợp

Thay đường dẫn C:\linear.txt và lỗi num_training trong bài gốc bằng make_regression:

```
import numpy as np
from sklearn import linear_model
import sklearn.metrics as sm
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=100, n_features=1, noise=15, random_state=42)

training_samples = int(0.6 * len(X))
X_train, y_train = X[:training_samples], y[:training_samples]
X_test, y_test = X[training_samples:], y[training_samples:]

reg_linear = linear_model.LinearRegression()
reg_linear.fit(X_train, y_train)
y_test_pred = reg_linear.predict(X_test)

plt.scatter(X_test, y_test, color='red')
plt.plot(X_test, y_test_pred, color='black', linewidth=2)
plt.show()

print("Regressor model performance:")
print("MAE =", round(sm.mean_absolute_error(y_test, y_test_pred), 2))
print("MSE =", round(sm.mean_squared_error(y_test, y_test_pred), 2))
print("R2 =", round(sm.r2_score(y_test, y_test_pred), 2))
```

3 Hồi quy tuyến tính (Linear Regression)

Hồi quy tuyến tính

Mô hình thống kê phân tích **quan hệ tuyến tính** giữa biến phụ thuộc và một hoặc nhiều biến độc lập. Khi biến độc lập thay đổi, biến phụ thuộc thay đổi tương ứng theo quy luật tuyến tính.

Trong học máy

Dự đoán giá trị số liên tục từ quan hệ tuyến tính đã học; dùng trong mô hình dự báo, tài chính, đánh giá rủi ro.

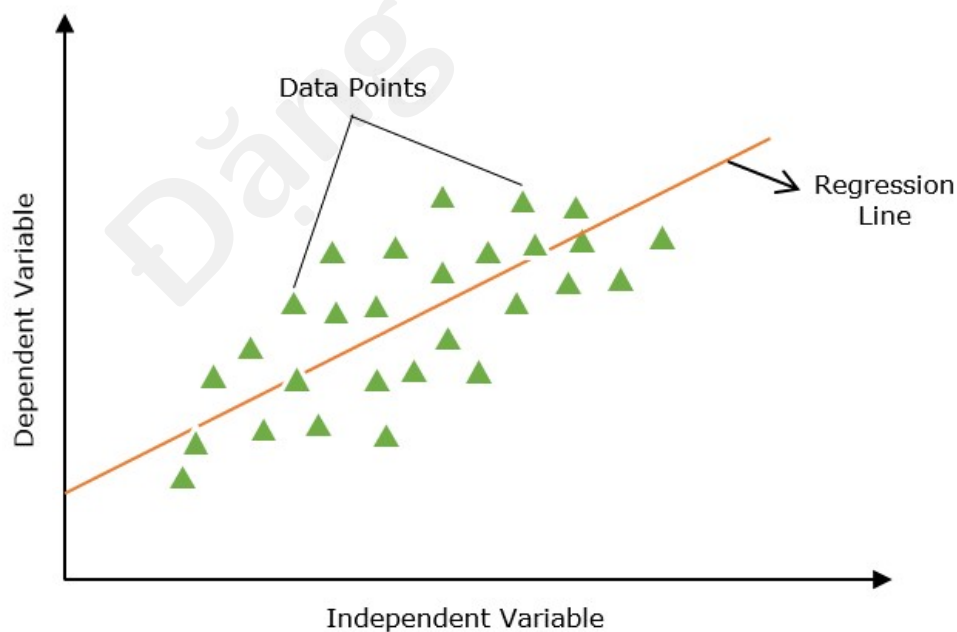
3.1 Dữ liệu ví dụ (một feature)

Bảng minh họa

Diện tích (ft², feature X) và giá nhà (target Y): mô hình dùng X để dự đoán Y .

Đường hồi quy

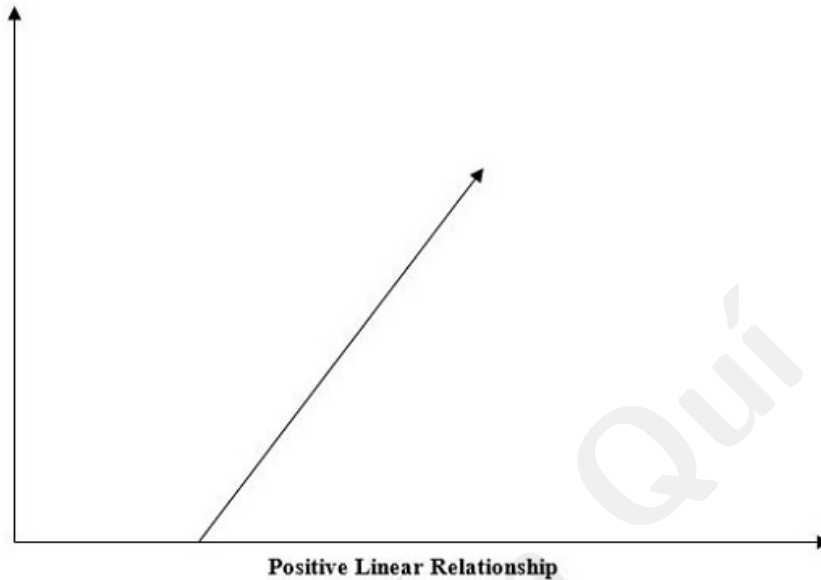
Đường thẳng thể hiện quan hệ giữa biến phụ thuộc và độc lập.



3.2 Quan hệ tuyến tính dương và âm

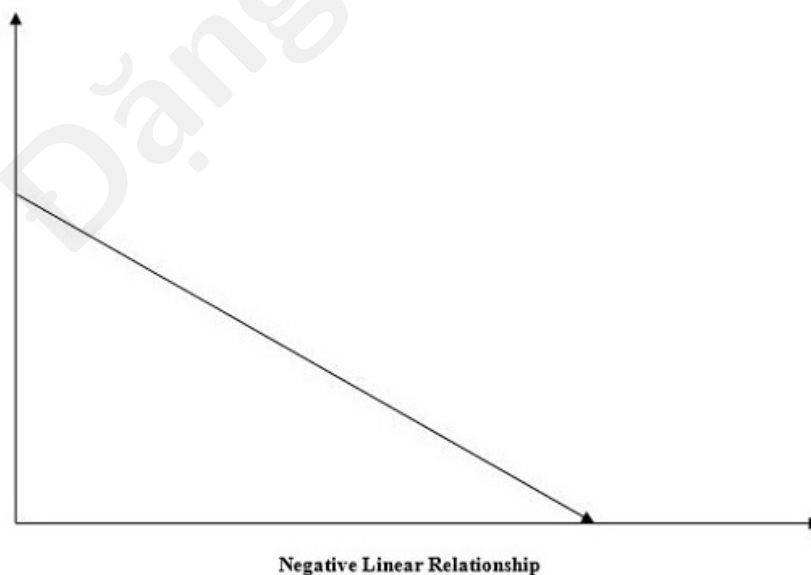
Quan hệ dương

Cả hai biến cùng tăng (hoặc cùng giảm).



Quan hệ âm

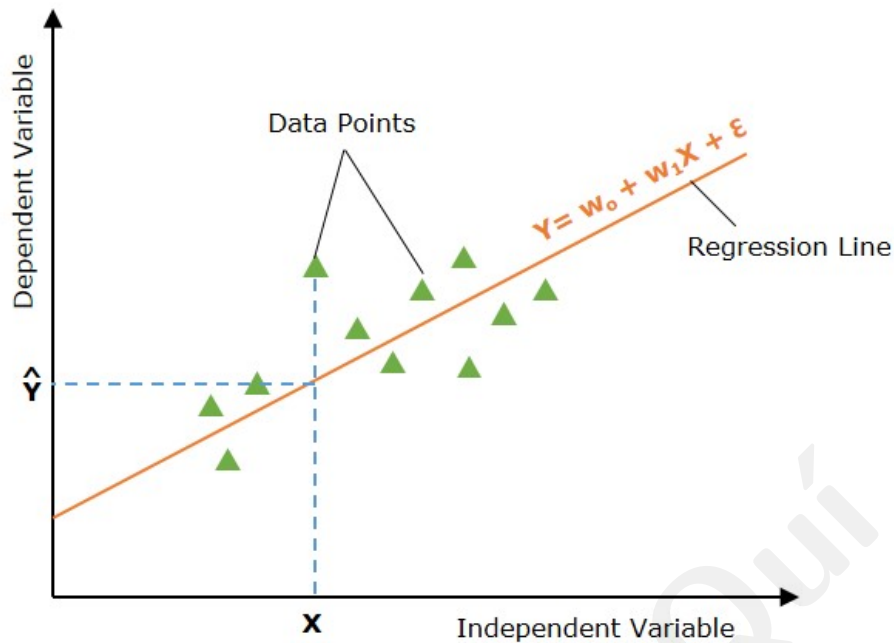
Biến độc lập tăng, biến phụ thuộc giảm (bài gốc ghi nhầm chữ “positive”).



3.3 Hai loại hồi quy tuyến tính

1. **Đơn biến** (Simple): một X .
2. **Đa biến** (Multiple): nhiều $X_1, \dots, X_p$.

3.3.1 Hồi quy tuyến tính đơn



$$\hat{Y} = w_0 + w_1X + \epsilon$$

- Y : target; X : feature; w_0 : hệ số chặn; w_1 : độ dốc; ϵ : sai số.

3.3.2 Hồi quy tuyến tính đa biến

$$\hat{Y} = w_0 + w_1X_1 + w_2X_2 + \dots + w_pX_p + \epsilon$$

3.4 Cách hoạt động

Ba bước chính

1. **Giả thuyết**: quan hệ tuyến tính giữa đầu vào và đầu ra.
2. **Hàm mất mát**: đo sai số dự đoán (MSE, MAE, ...).
3. **Tối ưu**: cập nhật tham số đến khi mất mát nhỏ.

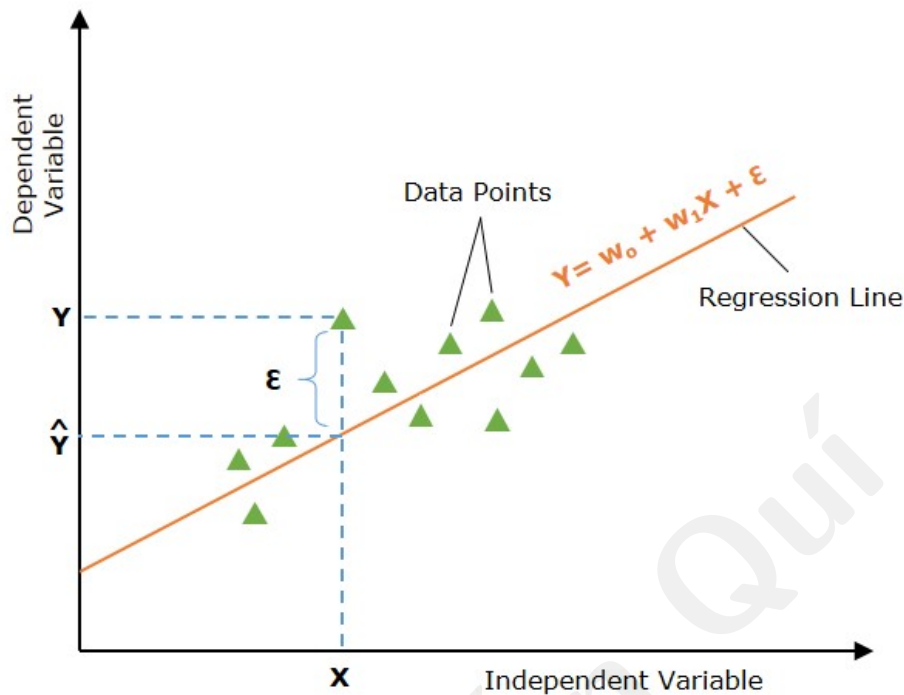
3.5 Hàm giả thuyết

$$\hat{Y} = w_0 + w_1X \quad (\text{đơn biến})$$

$$\hat{Y} = w_0 + w_1X_1 + \dots + w_pX_p \quad (\text{đa biến})$$

3.6 Đường khớp tốt nhất

Đường hồi quy tốt nhất khi sai số giữa giá trị thực và dự đoán là nhỏ nhất (trung bình).



3.7 Hàm mất mát

MSE (phổ biến)

$$J(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2, \quad \hat{Y}_i = w_0 + w_1 X_i$$

3.8 Gradient descent

Cập nhật tham số

$$w_0 \leftarrow w_0 - \alpha \frac{\partial J}{\partial w_0}, \quad w_1 \leftarrow w_1 - \alpha \frac{\partial J}{\partial w_1}$$

với α là learning rate. Lặp đến khi thay đổi w_0, w_1 không đáng kể.

3.9 Giả định mô hình

Giả định

- Ít hoặc không đa cộng tuyến.
- Ít tự tương quan trong phần dư.

- Quan hệ giữa response và feature là **tuyến tính**.

Vì phạm giả định có thể làm ước lượng chệch hoặc kém hiệu quả.

3.10 Metric đánh giá

R^2 , MSE, RMSE, MAE

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}, \quad \text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\text{MSE}}, \quad \text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

3.11 Ứng dụng, ưu điểm và thách thức

Ứng dụng

Mô hình dự báo (bất động sản), chọn feature, dự báo tài chính, quản trị rủi ro.

Ưu điểm

Dễ giải thích, huấn luyện nhanh, nền tảng phân tích dự đoán, đơn giản và hiệu quả tính toán.

Thách thức

Overfitting, đa cộng tuyến, ảnh hưởng outlier; với dữ liệu phi tuyến có thể cần **hồi quy đa thức**.

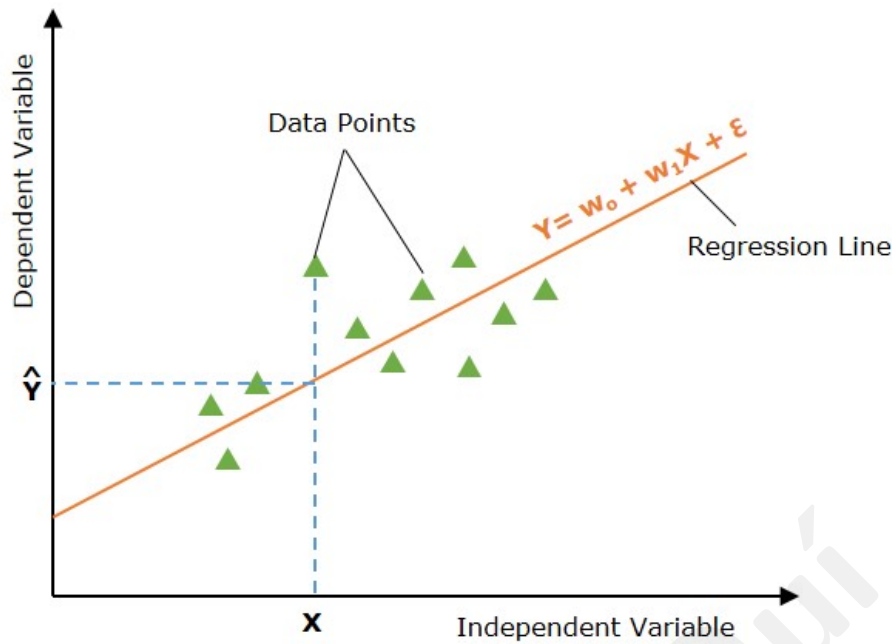
4 Hồi quy tuyến tính đơn (Simple Linear Regression)

Hồi quy tuyến tính đơn

Phương pháp học có giám sát dùng **một** biến độc lập để dự đoán biến phụ thuộc; mô hình quan hệ tuyến tính giữa hai biến. Khi thuật toán hồi quy tuyến tính chỉ có một feature $\Rightarrow$ hồi quy đơn; nhiều feature $\Rightarrow$ hồi quy đa biến.

Biến độc lập và phụ thuộc

- **Độc lập** (predictor): trục ngang; ví dụ số năm kinh nghiệm.
- **Phụ thuộc** (response): trục dọc; ví dụ lương.



Mô hình

$$Y = w_0 + w_1 X + \epsilon$$

4.1 Cách hoạt động

Tìm đường khớp tốt nhất

Giả thuyết: $\hat{Y} = w_0 + w_1 X$. Không gian giả thuyết là tập mọi đường thẳng với (w_0, w_1) khác nhau. Khởi tạo tham số, dự đoán, tính mất mát (thường MSE), tối ưu (thường gradient descent).

$$J(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

4.2 Giả định

Giả định cổ điển

- **Tuyến tính:** scatter plot gần đường thẳng.
- **Phương sai đồng nhất** (homoscedasticity).
- **Độc lập:** quan sát không phụ thuộc lẫn nhau.
- **Chuẩn phần dư:** histogram phần dư gần chuẩn.

4.3 Triển khai Python

Dataset Salary_Data.csv

30 mẫu: YearsExperience (feature), Salary (target). File: data/Salary_Data.csv.

Bước 1 — Chuẩn bị dữ liệu

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset = pd.read_csv('data/Salary_Data.csv')
X = dataset.iloc[:, :-1].values # YearsExperience
y = dataset.iloc[:, -1].values # Salary
print(dataset.head())
```

Kiểm tra tuyến tính

```
plt.scatter(X, y, color="green")
plt.title("Salary vs Experience")
plt.xlabel("Years of Experience")
plt.ylabel("Salary (INR)")
plt.show()
```



Chia train/test (80/20)

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)
```

Bước 2 — Huấn luyện

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

Bước 3 — Dự đoán trên tập kiểm tra

```
y_pred = regressor.predict(X_test)
df = pd.DataFrame({'Actual Values': y_test, 'Predicted Values': y_pred})
print(df)
```

Bước 4 — Đánh giá

```
from sklearn.metrics import mean_squared_error, mean_absolute_error,
    r2_score
import numpy as np

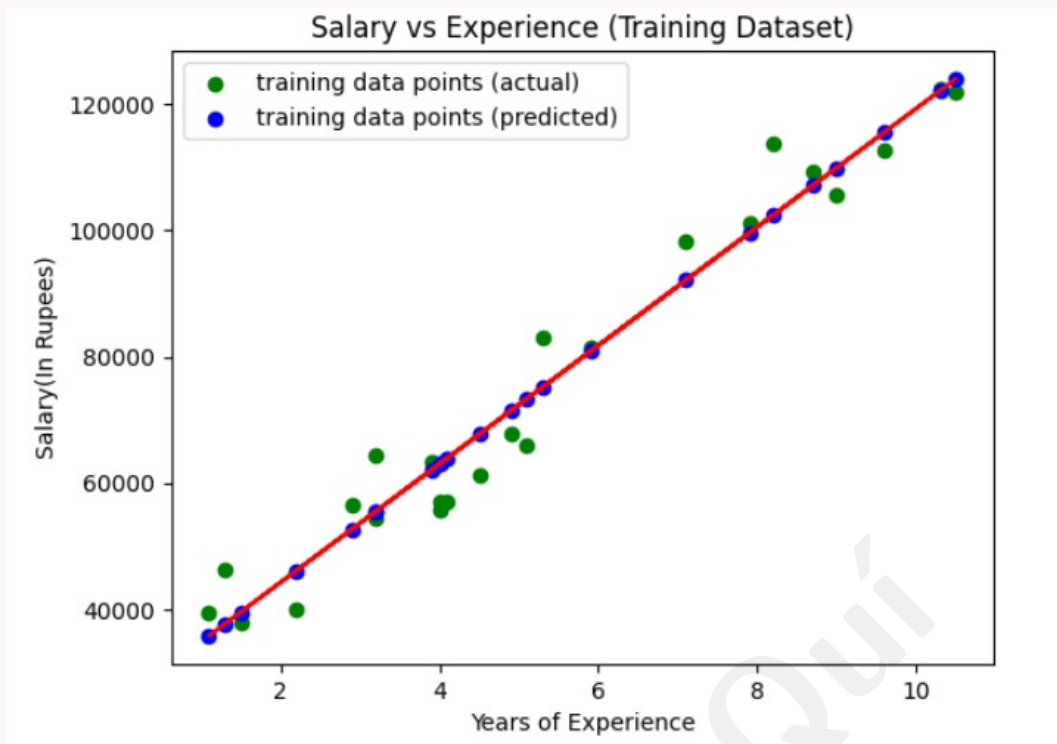
y_pred = regressor.predict(X_test)
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2:", r2_score(y_test, y_pred))
```

Output trong bài gốc

Bài gốc in MSE/MAE cực nhỏ do lỗi code (dùng `mean_squared_error` cho cả RMSE và có thể nhầm tập). Code trên dùng `np.sqrt` cho RMSE là đúng.

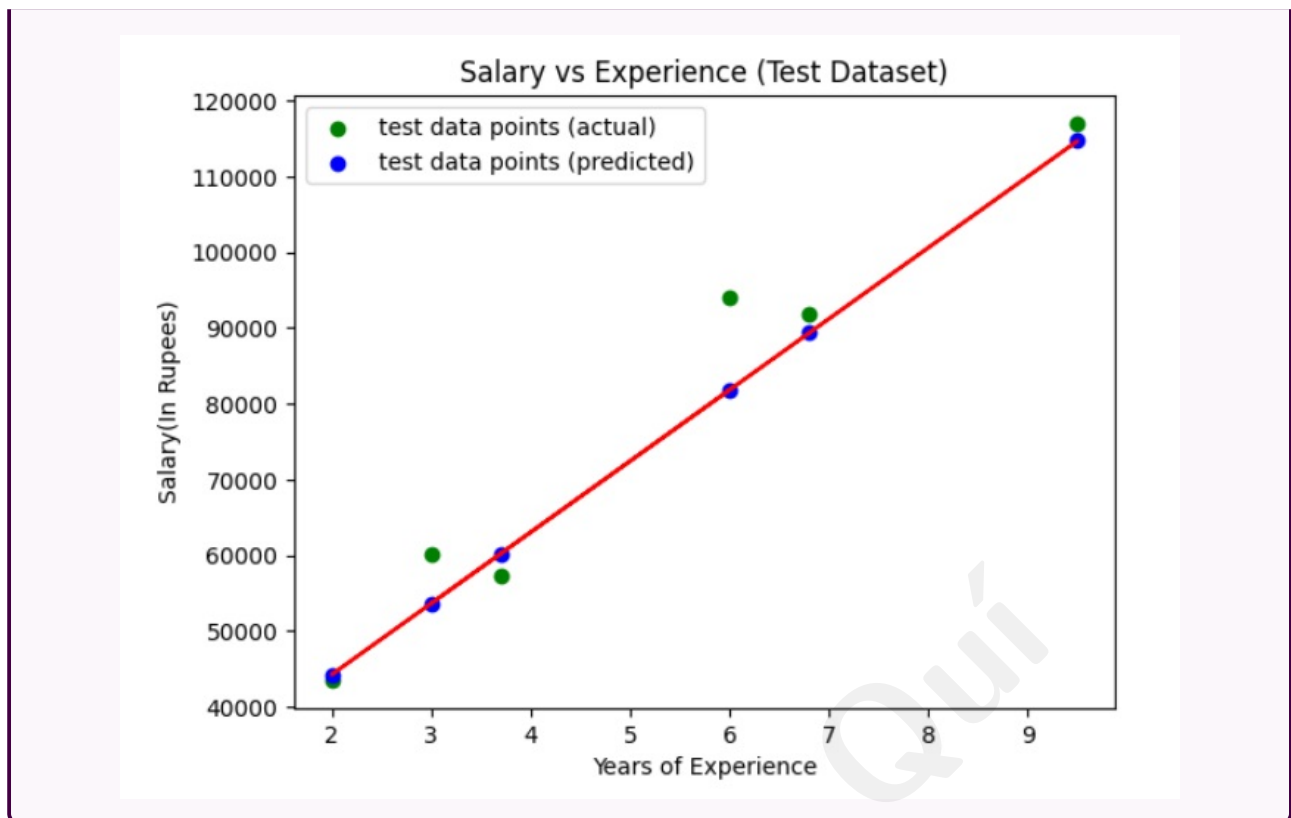
Bước 5 — Trực quan tập huấn luyện

```
y_pred_train = regressor.predict(X_train)
plt.scatter(X_train, y_train, color="green", label="actual")
plt.scatter(X_train, y_pred_train, color="blue", label="predicted")
plt.plot(X_train, y_pred_train, color="red")
plt.xlabel("Years of Experience")
plt.ylabel("Salary (INR)")
plt.legend()
plt.show()
```



Bước 6 — Trực quan tập kiểm tra

```
y_pred = regressor.predict(X_test)
plt.scatter(X_test, y_test, color="green", label="test actual")
plt.scatter(X_test, y_pred, color="blue", label="test predicted")
plt.plot(X_test, y_pred, color="red")
plt.legend()
plt.show()
```

5 Hồi quy tuyến tính đa biến (Multiple Linear Regression)

Hồi quy tuyến tính đa biến (MLR)

Thuật toán học có giám sát mô hình quan hệ giữa **một** biến phụ thuộc liên tục và **nhiều** biến độc lập. Biến phân loại (ví dụ **State**) cần mã hóa số trước khi đưa vào mô hình tuyến tính.

So với hồi quy đơn

MLR là mở rộng của hồi quy đơn: dự đoán bằng hai hoặc nhiều feature.

Phương trình

Với n quan sát, p feature:

$$h(x_i) = w_0 + w_1x_{i1} + w_2x_{i2} + \dots + w_px_{ip}$$

Có phần dư:

$$y_i = h(x_i) + e_i, \quad e_i = y_i - h(x_i)$$

5.1 Giả định

Giả định MLR

1. Tuyến tính giữa target và predictor.
2. Các quan sát độc lập.
3. Phương sai phần dư đồng nhất (homoscedasticity).
4. Phần dư gần phân phối chuẩn.
5. Không đa cộng tuyến mạnh giữa feature.
6. Không tự tương quan phần dư.
7. Giá trị biến độc lập cố định giữa các mẫu lặp.

5.2 Triển khai Python

Dataset data.csv

50 dòng: R&D Spend, Administration, Marketing Spend, State, Profit. Ví dụ dưới bỏ State như bài gốc (đơn giản hóa).

Bước 1 — Chuẩn bị

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

dataset = pd.read_csv('data/data.csv')
X = dataset[['R&D Spend', 'Administration', 'Marketing Spend']]
y = dataset['Profit']
print(X.head())
print(y.head())
```

Chia train/test (20% test)

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)
```

Bước 2 — Huấn luyện

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

Bước 3 — Kiểm tra

```
y_pred = regressor.predict(X_test)
df = pd.DataFrame({'Real Values': y_test.values,
                   'Predicted Values': y_pred})
print(df)
```

Bước 4 — Metric

```
from sklearn.metrics import mean_squared_error, mean_absolute_error,
    r2_score
import numpy as np

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("R2:", r2)
```

Đọc $R^2 \approx 0,96$ (bài gốc)

Khoảng 96% biến thiên của Profit được giải thích bởi ba feature đầu vào (trên tập kiểm tra của ví dụ gốc).

Bước 5 — Dự đoán điểm mới

```
new_data = [[166343.2, 136787.8, 461724.1]]
profit = regressor.predict(new_data)
print(profit)
```

Hệ số và chặn

```
print("coefficients:", regressor.coef_)
print("intercept:", regressor.intercept_)
```

Kiểm tra tay: Profit $\approx w_0 + w_1X_1 + w_2X_2 + w_3X_3$.

5.3 Ứng dụng và thách thức**Ứng dụng**

Tài chính, marketing, bất động sản, y tế, kinh tế, khoa học xã hội.

Thách thức

Đa cộng tuyến, overfitting/underfitting, phi tuyến, outlier, dữ liệu thiếu.

5.4 So sánh đơn và đa biến

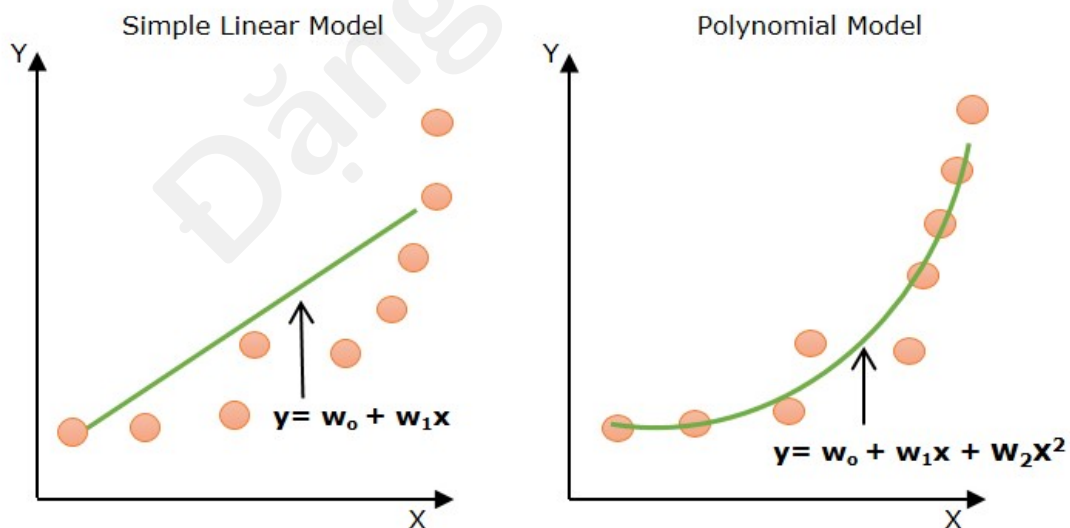
Tiêu chí	Đơn biến	Đa biến
Số biến độc lập	1	≥ 2
Phương trình	$y = w_0 + w_1x$	$y = w_0 + \sum w_jx_j$
Độ phức tạp	Thấp hơn	Cao hơn
Giải thích hệ số	Dễ hơn	Khó hơn

6 Hồi quy đa thức (Polynomial Regression)**Hồi quy đa thức**

Mô hình quan hệ giữa biến độc lập và phụ thuộc bằng **đa thức bậc n** , vượt qua giới hạn tuyến tính của hồi quy đơn/đa biến tuyến tính thuần.

Khi nào dùng?

Dữ liệu **phi tuyến** không khớp đường thẳng; hồi quy tuyến tính bỏ sót pattern cong.



6.1 Phương trình

Đa thức bậc n

$$y = w_0 + w_1x + w_2x^2 + \dots + w_nx^n + \epsilon$$

Bậc 2 (parabol):

$$y = w_0 + w_1x + w_2x^2 + \epsilon$$

6.2 Cách hoạt động

Biến đổi feature

Tạo feature $x, x^2, \dots, x^n$ từ x gốc, rồi huấn luyện `LinearRegression` trên không gian mở rộng — tương tự MLR nhưng các cột đa thức phụ thuộc cùng một x đầu vào.

6.3 Triển khai Python

ice_cream_selling_data.csv

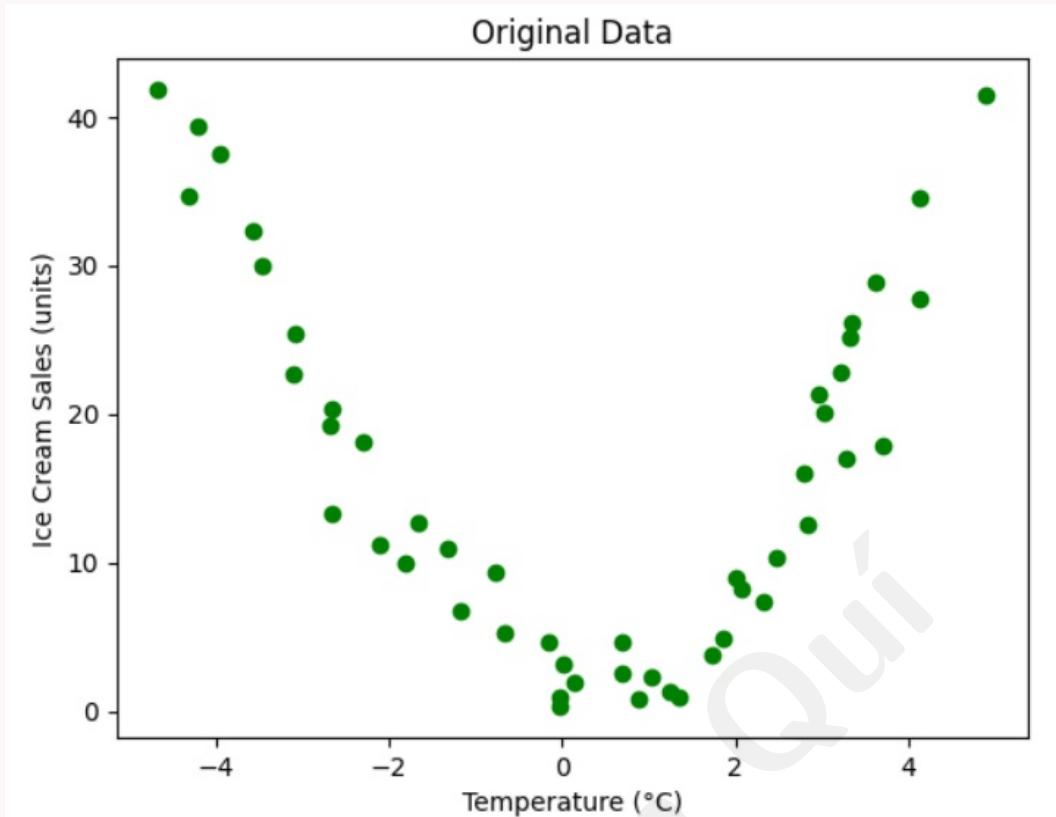
49 mẫu: Temperature, IceCreamSales. File: data/ice_cream_selling_data.csv.

Bước 1 — Nạp và trực quan

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures

data = pd.read_csv('data/ice_cream_selling_data.csv')
X = data.iloc[:, 0].values.reshape(-1, 1)
y = data.iloc[:, 1].values

plt.scatter(X, y, color="green")
plt.xlabel("Temperature (C)")
plt.ylabel("Ice Cream Sales (units)")
plt.title("Original Data")
plt.show()
```



Đặc trưng đa thức bậc 2

```
degree = 2
poly_features = PolynomialFeatures(degree=degree)
X_poly = poly_features.fit_transform(X)
```

Bước 2 — Huấn luyện

```
from sklearn.linear_model import LinearRegression
lr_model = LinearRegression()
lr_model.fit(X_poly, y)
```

Bước 3 — Dự đoán

```
y_pred = lr_model.predict(X_poly)
df = pd.DataFrame({'Actual Values': y, 'Predicted Values': y_pred})
print(df.head())
```

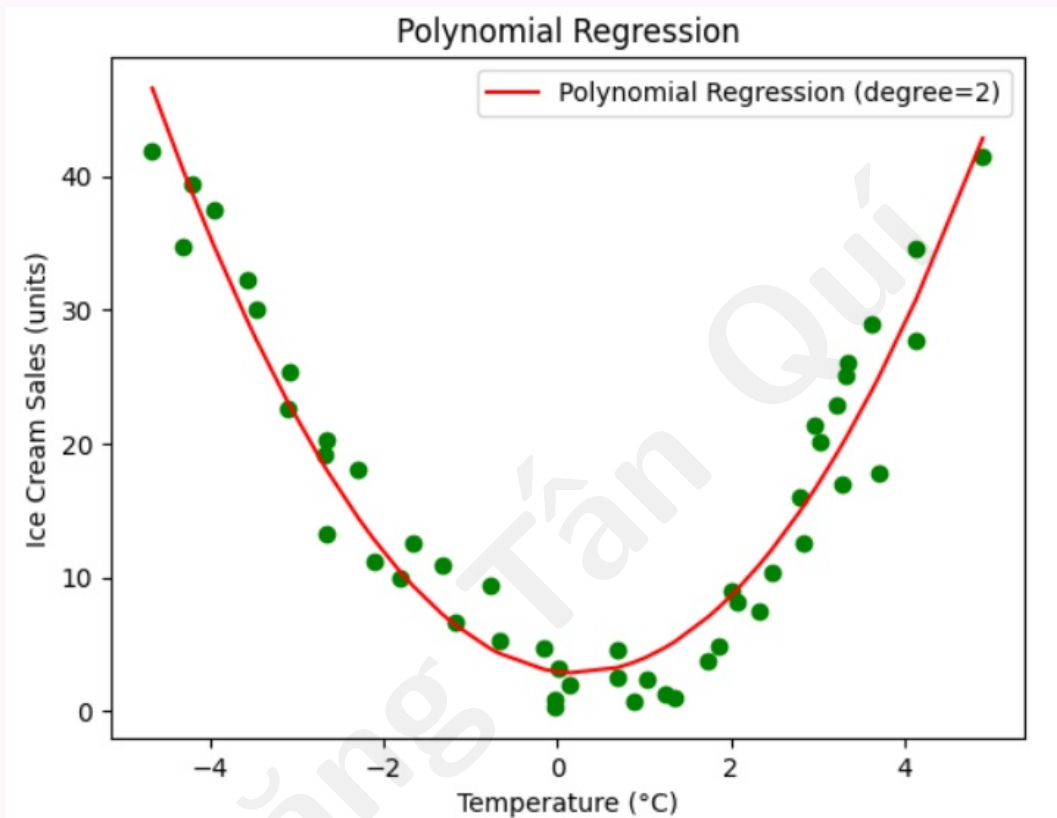
Bước 4 — R^2

```
from sklearn.metrics import r2_score
print("R2:", r2_score(y, y_pred))
```

Bài gốc: $R^2 \approx 0,932$ — khoảng 93% biến thiên được giải thích.

Bước 5 — Vẽ đường cong hồi quy

```
plt.scatter(X, y, color="green")
plt.plot(X, y_pred, color='red',
         label=f'Polynomial Regression (degree={degree})')
plt.xlabel("Temperature (C)")
plt.ylabel("Ice Cream Sales (units)")
plt.legend()
plt.show()
```

**Dự đoán nhiệt độ mới**

```
X_new = np.array([[1.9929]])
X_new_poly = poly_features.transform(X_new)
print(lr_model.predict(X_new_poly))
```

Bậc đa thức

Bậc quá cao dễ overfitting; nên thử cross-validation hoặc regularization khi mở rộng thực tế.